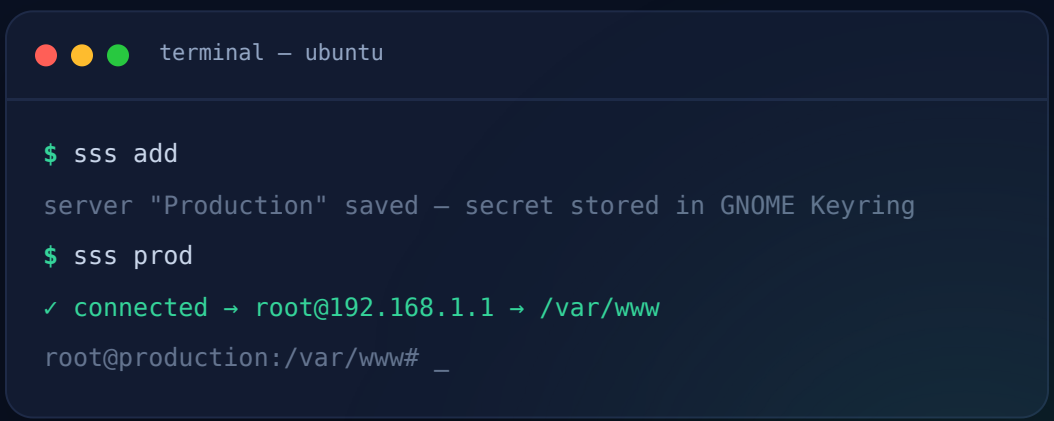
SSH Desk · Version 1.0 · Guide de l'utilisateur

Gestionnaire professionnel de connexions SSH pour Ubuntu

Guide de l'utilisateur

SSH Desk

De l'enregistrement d'un serveur à un vrai shell interactif — en une seule commande. Interface graphique, ligne de commande et noyau partagé ; les mots de passe ne vivent que dans GNOME Keyring et le moteur SSH est l'OpenSSH de votre propre système.



Terminal PTY réel

Secrets uniquement dans le trousseau

Gestionnaire de fichiers SFTP

Tunnels Local/Remote/SOCKS

Installation hors ligne via miroirs

Édition 1.0 — novembre 2026

github.com/hmplus28/SSH-Desk

Sommaire

Dans ce guide

01 Découvrir SSH Desk

02 Installation et configuration

03 Démarrage rapide : votre premier serveur

04 Connexion et gestion des chemins

05 Référence complète du CLI sss

06 L'interface graphique

07 Le vrai terminal (PTY)

08 Gestion des fichiers avec SFTP

09 Clés, tunnels et commandes

10 Le modèle de sécurité

11 Sauvegardes et dépannage

Chapitre 01

Découvrir SSH Desk

SSH Desk est un vrai produit installable pour gérer tous vos serveurs SSH sous Ubuntu — pas une démonstration. Trois modules — l'interface graphique, la ligne de commande

et le noyau partagé — travaillent exactement sur une même base de données et un même magasin de secrets : enregistrez un serveur dans l'interface et connectez-vous depuis n'importe quel terminal avec `sss prod` dans la foulée, même quand l'interface graphique n'est pas du tout lancée.

Le moteur de connexion est l'**OpenSSH du système lui-même** ; le protocole SSH n'est pas réimplémenté, le comportement de connexion est donc exactement celui d'Ubuntu. Le terminal intégré est un **vrai PTY** : couleurs, flèches, **Ctrl+C**, vim et nano fonctionnent correctement. Les mots de passe sont stockés uniquement dans **GNOME Keyring** et n'atteignent jamais la base de données, les journaux, les arguments de commande ni les variables d'environnement.

GUI — interface de bureau

PySide6 avec thèmes sombre/clair, 7 langues d'interface avec RTL complet pour le persan et l'arabe, terminal multi-onglets et gestionnaire de fichiers.

CLI — ligne de commande

La commande `sss` fonctionne sans l'interface graphique ; le nom de commande est configurable ; complétion bash/zsh incluse.

Core — noyau partagé

SQLite + SQLAlchemy avec migrations, le service Keyring, le constructeur de commandes ssh et la politique de Host Key.

3

modules, un moteur

9

pages dans l'interface

30+

sous-commandes CLI

Chapitre 02

Installation et configuration

› Prérequis

```
# Ubuntu 22.04 / 24.04+
sudo apt update
sudo apt install -y openssh-client gnome-keyring libsecret-1-0 \
     python3 python3-venv python3-pip
```

› Option A — installation au niveau utilisateur (recommandée, sans sudo)

```
git clone https://github.com/hmplus28/SSH-Desk.git
cd SSH-Desk
bash scripts/install.sh
```

Le script vérifie Python et OpenSSH, crée un environnement virtuel dans `~/.local/share/ssh-desk/venv`, installe le paquet avec toutes ses dépendances et crée dans `~/.local/bin` les raccourcis `ssh-desk` (interface graphique), `ssh-desk-cli`, `ssh-desk-askpass` et `sss` ; la complétion bash et l'icône de l'application sont installées aussi.

› Option B — paquet DEB à l'échelle du système

```
bash scripts/package_deb.sh
sudo apt install ./dist/ssh-desk_1.0.0_amd64.deb
```

Le DEB installe un ensemble PyInstaller autonome sous `/opt/ssh-desk` et enregistre l'entrée de menu et l'icône système. La mise à jour se fait en installant le nouveau paquet par-dessus ; vos données et vos secrets restent intacts.

› Internet faible ou coupé : miroirs et installation hors ligne

Si votre réseau est instable ou si PyPI est difficile à joindre, pas d'inquiétude : chaque appel pip des scripts d'installation essaie automatiquement une **chaîne de miroirs** — un miroir injoignable est ignoré sans aucune action de votre part et le suivant est tenté. L'ordre par défaut est **PyPI officiel → ArvanCloud (Iran) → TUNA → Aliyun → Tencent**.

```
# force a specific working mirror
bash scripts/install.sh --mirror-name ir    # ArvanCloud (Iran)
bash scripts/install.sh --mirror-name tuna  # TUNA (China)

# 1) download everything once (on any working network)
bash scripts/download_wheels.sh --mirror-name ir
# 2) install later with NO network at all
bash scripts/install.sh --offline
```

En mode `--offline`, pip n'a le droit à aucune connexion réseau et n'installe que depuis le dossier `vendor/wheels` — une coupure d'Internet en pleine installation ne peut donc plus laisser le travail à moitié fait. La même chaîne de miroirs est utilisée par `scripts/build.sh`, la construction du DEB est donc protégée des coupures elle aussi.

› Où se trouvent les fichiers importants ?

Emplacement	Chemin
Base de données de l'application	`~/.local/share/ssh-desk/ssh-desk.db`
Paramètres	`~/.config/ssh-desk/config.toml`
known_hosts de l'application	`~/.local/share/ssh-desk/known_hosts`
Clés générées	`~/.local/share/ssh-desk/keys/`
Journaux	`~/.local/share/ssh-desk/logs/`
Mots de passe et secrets	GNOME Keyring (Secret Service)

Désinstallation : `bash scripts/uninstall.sh` demande explicitement votre accord avant de supprimer données et secrets ; sans votre confirmation rien n'est effacé.

Chapitre 03

Démarrage rapide : votre premier serveur

Étape 1 — lancement : exécutez `ssh-desk` pour ouvrir l'interface graphique ; ou restez entièrement dans le terminal et utilisez `sss add`.

Étape 2 — ajouter un serveur : sur la page Servers, cliquez « Add server ». Saisissez le nom, l'alias (unique — c'est avec lui que vous vous connectez), l'hôte, le port, l'utilisateur et le chemin par défaut. Si vous choisissez l'authentification par mot de passe, le secret est stocké aussitôt dans GNOME Keyring — pas dans la base de données. Avant d'enregistrer, le bouton **Test** effectue une vraie connexion pour que vous sachiez que tout est correct.

```
# the same thing from the terminal (interactive wizard)
sss add
sss add --alias prod --host 192.168.1.1 --port 22 --user root \
        --auth password --path /var/www
```

Étape 3 — connexion : depuis n'importe quel terminal, tapez `sss prod` pour ouvrir un shell interactif au chemin par défaut. Dans l'interface graphique, un double-clic sur le serveur suffit pour ouvrir un vrai onglet de terminal.

> **Première connexion :** l'empreinte de la clé d'hôte est affichée ; comparez-la avec l'administrateur du serveur et acceptez-la une fois. Si la clé d'hôte **change** plus tard, la connexion est refusée immédiatement et des conseils pour écarter une attaque MITM sont affichés.

Chapitre 04

Connexion et gestion des chemins

Chaque serveur peut avoir un **chemin par défaut** et autant de **chemins enregistrés nommés** que voulu ; par exemple définissez une fois `website → /var/www/html` et `nginx → /etc/nginx` pour le serveur de production et sautez toujours dans ce dossier avec un seul mot :

```
sss prod               # shell at the server's default path
sss prod nginx         # shell at the saved path named "nginx"
sss prod /var/www      # shell at any absolute path
sss prod --exec "uptime"           # run one command, no shell
sss prod --user ubuntu --port 2222 # per-run override (NOT saved)
sss prod --dry-run                 # print the exact ssh command
```

Si le chemin demandé n'existe pas sur le serveur, le message d'erreur est clair et compréhensible — pas une déconnexion vague. Les chemins sont transmis avec un

guillemetage sûr (shlex), si bien que les espaces ou caractères spéciaux dans les noms de dossiers ne posent aucun problème. Les options `--user` et `--port` ne s'appliquent qu'à cette exécution et ne sont jamais enregistrées.

› Codes de sortie du CLI

Code	Signification
`0`	Opération réussie
`1`	Erreur d'exécution (connexion, réseau, Keyring, …)
`2`	Entrée non valide ou mauvais usage de la commande
`130`	Interrompu par Ctrl+C

Ces codes rendent `sss` fiable dans vos scripts et vos automatisations ; vous pouvez par exemple vérifier en CI ou en cron que la connexion a réussi.

Chapitre 05

Référence complète du CLI sss

Le nom de commande `sss` n'est qu'un exemple ; renommez-le avec `sss config cli-name cnc` comme vous voulez (cnc, sshgo, …) — le changement s'applique à tout le système (un nouveau raccourci dans `~/.local/bin`). Tout premier jeton qui n'est pas une sous-commande connue est interprété comme un alias de serveur : `sss prod` équivaut à `sss connect prod`.

› Serveurs

Commande	Ce qu'elle fait
`sss list`	Tableau des serveurs ; filtres `--group --favorites --search`
`sss add`	Ajout interactif ; chaque champ peut aussi être passé directement
`sss info prod`	Détails complets ; `--show-secret` exige de taper l'alias
`sss edit prod`	Modifier les champs ; `--set-password` enregistre un nouveau mot de passe
`sss delete prod`	Suppression avec confirmation ; le secret du trousseau est purgé aussi
`sss search TERME`	Rechercher dans nom, alias, hôte, utilisateur et description

› Surveillance et configuration

Commande	Ce qu'elle fait
`sss history`	Historique des connexions avec filtres serveur/limite/statut
`sss current`	Sessions SSH actives et tunnels en cours
`sss config list`	Tous les paramètres et emplacements de fichiers
`sss config set KEY VALUE`	Modifier un paramètre
`sss config cli-name cnc`	Renommer la commande à l'échelle du système
`sss config completions`	Installer la complétion bash/zsh

› Clés, tunnels et transfert de configuration

Commande	Ce qu'elle fait
`sss keys generate`	Générer Ed25519/RSA/ECDSA au format OpenSSH
`sss keys import`	Enregistrer une clé existante (avec `--copy` vers le magasin en 0600)
`sss keys export-pub`	Exporter la clé publique
`sss tunnel start|stop`	Exécuter un tunnel en daemon détaché du terminal
`sss import --file ~/.ssh/config`	Importer ssh config avec aperçu et détection des doublons
`sss export FILE.toml`	Sauvegarde ; secrets seulement avec `--include-secrets`
`sss restore FILE.toml`	Restaurer une sauvegarde
`sss install-cli`	Reconstruire les raccourcis de terminal

Complétion : celle de bash est installée automatiquement (sous-commandes + alias de serveurs). Pour zsh : `sss config completions --shell zsh --install`, puis ajoutez le chemin completions à `~/.zshrc`.

Chapitre 06

L'interface graphique

La fenêtre principale comporte une barre latérale repliable avec 9 pages et une barre d'état (sessions et tunnels actifs). L'interface est disponible en 7 langues — anglais, persan, arabe, chinois, russe, espagnol et français — avec une vraie disposition de droite à gauche pour le persan et l'arabe ; le thème sombre/clair se change dans Settings.

Page	Rôle
Dashboard	Vue d'ensemble des serveurs, sessions et tunnels actifs
Servers	Recherche, filtres groupe/étiquette/favori, duplication, test rapide
Ajout/Modification	Quatre onglets : général, authentification, connexion et chemins, avancé (hôte de rebond et options ssh)
Détails du serveur	Détails ; le mot de passe est masqué et ne se révèle qu'après avoir tapé l'alias
Terminal	Sessions PTY réelles multi-onglets
Files	Gestionnaire de fichiers SFTP à deux volets avec transfert entre serveurs
Clés SSH / Redirection / Commandes / Historique	Gérer clés, tunnels, commandes enregistrées et historique des connexions
Settings	Nom de commande CLI, thème, langue, valeurs par défaut, état du trousseau, sauvegardes et import ssh-config

› Raccourcis clavier

`Ctrl+1..9` changer de page · `Ctrl+N` nouveau serveur · `Ctrl+K` nouvelle session de terminal · `Ctrl+Q` quitter.

> **GUI et CLI toujours synchronisées :** les deux interfaces partagent une base de données, un trousseau et des paramètres ; un serveur créé dans l'interface graphique est aussitôt connu de `sss` dans tous les terminaux — et inversement.

Chapitre 07

Le vrai terminal (PTY)

Chaque onglet de terminal est une vraie session PTY par-dessus OpenSSH — pas une boîte de texte. Tout ce qui est possible dans un vrai terminal est possible ici :

- Couleurs, flèches, complétion Tab, et exécution de **vim**, **nano** et **htop** fonctionnent correctement.

- **Ctrl+C** et **Ctrl+D** interrompent et ferment les processus exactement comme un terminal système.
- **Copier :** sélectionnez le texte à la souris (copie automatique) ou utilisez la barre d'outils ; **coller :** clic milieu, la barre d'outils ou `Ctrl+V` dans le terminal.
- **Redimensionnement :** déplacer la fenêtre informe instantanément le programme côté serveur (SIGWINCH) — les éditeurs et htop se redessinent correctement.
- Chaque onglet affiche le nom du serveur et le titre de fenêtre côté serveur ; pour vous reconnecter, ouvrez simplement une nouvelle session du même serveur.

> **Confidentialité :** le contenu du terminal n'est jamais journalisé ; seuls l'état de connexion et sa durée sont consignés dans l'historique.

Chapitre 08

Gestion des fichiers avec SFTP

Sur la page Files, choisissez un serveur et cliquez Connect : le volet gauche est votre système local, le droit est le serveur. Navigation par double-clic sur les dossiers, le bouton « monter » ou en éditant le chemin directement. Toutes les opérations du quotidien sont prises en charge :

- **Envoi et téléchargement** par glisser-déposer ou via les boutons.
- **Nouveau dossier, renommer, supprimer** (avec confirmation, récursif).
- **Changer les permissions** en forme octale (ex. 0644) — sans calcul mental.
- **Éditer les fichiers texte :** le fichier s'ouvre localement et, après enregistrement, un renvoi est proposé.
- **« Envoyer vers le serveur… » :** déplacez un fichier directement entre deux serveurs enregistrés — sans le télécharger via votre machine.

> **Si SFTP échoue mais que le terminal fonctionne :** le serveur a peut-être désactivé le sous-système SFTP (`Subsystem sftp` dans sshd_config) ou la clé d'hôte n'est pas encore acceptée — connectez-vous une fois via le terminal ou cliquez Test.

Chapitre 09

Clés, tunnels et commandes enregistrées

› Clés SSH

Générez une clé Ed25519 (recommandée), RSA ou ECDSA au format OpenSSH standard ; la phrase secrète optionnelle de la clé est stockée dans le trousseau elle aussi. Enregistrez des clés existantes (sans copier le fichier) ou, sur votre choix explicite, copiez-les dans le magasin de l'application avec les permissions 0600. La colonne d'empreinte aide à la vérification, celle des « serveurs associés » montre qui utilise chaque clé, et « Installer sur le serveur » construit la commande `ssh-copy-id` et l'exécute dans une session de terminal.

```
sss keys generate --name deploy --type ed25519
sss keys import --path ~/.ssh/id_ed25519 --name personal
sss keys export-pub deploy
```

› Tunnels de ports

Trois types de tunnels : **Local** (ex. `127.0.0.1:5432 → db-internal:5432` pour joindre une base interne), **Distant** (inverse — exposer un port d'un service local sur le serveur) et **Dynamique** (un proxy SOCKS5 tel que `127.0.0.1:1080`). Ils tournent dans un daemon indépendant — ils survivent à la fermeture de l'interface ou du CLI et leur état est visible dans `sss current`.

```
sss tunnel db       # start the tunnel named "db"
sss tunnel stop db
sss current         # live sessions + running tunnels
```

› Commandes enregistrées

Enregistrez les commandes fréquentes par serveur ou globalement (nom, commande, catégorie). Les commandes sensibles marquées « confirmation requise » demandent avant de s'exécuter. Lancez-les depuis la page Commands de l'interface ou avec `sss prod --exec "<command>"` dans le terminal.

Chapitre 10

Le modèle de sécurité

SSH Desk est conçu pour conserver les identifiants de serveurs de production et repose du début à la fin sur un principe : **les secrets ne vivent que dans le Secret Service**. Les contrôles les plus importants :

1 — Mots de passe uniquement dans GNOME Keyring

La base de données ne garde qu'une référence sans secret (JSON) ; le mot de passe lui-même n'atteint jamais SQLite (vérifié par des tests automatisés).

2 — askpass sécurisé, pas sshpass

OpenSSH s'exécute avec `SSH_ASKPASS_REQUIRE=force` et un assistant sécurisé qui lit le secret « par référence » depuis le trousseau ; le mot de passe n'apparaît jamais dans argv, l'environnement, les journaux ou les fichiers temporaires.

3 — Clés d'hôte toujours strictes

`StrictHostKeyChecking=yes` est définitif dans le constructeur de commandes et impossible à désactiver ; hôte inconnu → dialogue d'empreinte ; clé modifiée → alerte MITM et conseils `ssh-keygen -R`.

4 — Résistant aux injections

Tous les appels subprocess utilisent des tableaux d'arguments sans shell=True ; les chemins distants sont guillemés avec shlex.quote et toutes les entrées sont validées par Pydantic.

› Autres contrôles

- Permissions des fichiers : base et configuration 0600, dossiers de données 0700, clés privées 0600.
- Le journaliseur nettoie les motifs `password=`, `secret=` et les secrets enregistrés à l'exécution.
- Révéler/copier un mot de passe exige de taper l'alias du serveur ; le presse-papiers s'efface automatiquement après 30 secondes.

- Les opérations sensibles — suppression d'un serveur, suppression récursive de fichiers, purge totale des données (taper PURGE) — demandent toujours confirmation.
- Aucun daemon ne tourne en root ; l'interface graphique et le CLI n'ont jamais besoin de sudo.
- La génération de clés est entièrement intra-processus ; la phrase secrète n'atteint jamais argv ni les fichiers temporaires.
- Export/sauvegarde sans secrets par défaut ; inclure les secrets est optionnel avec un avertissement explicite.

Chapitre 11

Sauvegardes, transfert et dépannage

› Sauvegarde et restauration

```
sss backup backup.toml                # no secrets by default
sss restore backup.toml
sss export full.toml --include-secrets  # opt-in + warning
sss import --file ~/.ssh/config         # preview, then: --apply
```

L'import depuis `~/.ssh/config` se fait avec aperçu et détection des serveurs en doublon ; l'export TOML sert à transférer la configuration vers une autre machine et ne contient aucun secret tant que vous ne le demandez pas explicitement.

› Erreurs courantes et solutions

Symptôme	Solution
`sss: command not found`	Ajoutez `~/.local/bin` au PATH ou lancez `sss install-cli`
« Secret Service is not available »	`sudo apt install gnome-keyring libsecret-1-0` puis déconnectez/reconnectez une fois
Le trousseau est verrouillé	Déverrouillez-le une fois ; l'application demande aussi de façon interactive quand c'est possible
Host key changed	Seulement après vérification : `ssh-keygen -R "[host]:port"` puis acceptez à nouveau
Permission denied	Mot de passe : `sss edit prod --set-password` · Clé : `sss keys list`
L'interface ne s'ouvre pas	`ssh-desk 2>&1 | tail -20` ; installez les bibliothèques Qt ; Wayland : `QT_QPA_PLATFORM=xcb`
Port de tunnel occupé	`ss -tlnp | grep <port>` puis `sss tunnel stop`

> **Réinitialisation complète :** `sss uninstall-data` — vous devez taper PURGE pour effacer les données et les secrets du trousseau.

Connexions sûres, toujours à portée de main.

SSH Desk est un logiciel libre sous licence MIT. Si ce guide vous a été utile, rendez-vous sur le dépôt GitHub pour signaler des problèmes ou proposer des fonctionnalités.

`github.com/hmplus28/SSH-Desk`

MIT License